

AI Learning & Study Assistant

An Agentic AI-Based Personalized Learning System

College Code : 9530

College Name : St.Mother Theresa Engineering College

Department : Computer Science and Engineering

REG.NO : 953023104068

Student NM ID : 29FE281ACC1826EBE0C4C72E93815F9C

Student Name : Murugeshwari R

AI Learning & Study Assistant

An Agentic AI-Based Personalized Learning System

Executive Project Summary

Attribute	System Implementation Details
Project Name	AI Learning & Study Assistant
Project Nature	Agentic AI / Intelligent Academic Learning System

Primary Developer	Murugeshwari R – sakthi830000@gmail.com
Core Language	Python 3.8+ (Pure Standard Library Implementation)
User Interface	Command Line Interface (CLI / Interactive Numbered Menu)
System Architecture	Decoupled Modular Tool-Based Agent Architecture
Information Retrieval	Lightweight RAG (Paragraph Chunking, Stemming, TF Cosine Similarity)
Data Persistence	Normalized Flat-File JSON Storage (data/*.json)
Course Coverage	5 Courses (Python, Java, Database/SQL, AI/ML, Web Development)

Table of Contents

1. Project Overview
2. Objectives & Proposed Solution
3. Key Features & Capabilities
4. Implementation & System Design
5. Working Process & Operational Workflows
6. Terminal Output Execution Screenshots

7. Results Achieved
8. Challenges Faced & Engineering Solutions
9. System Limitations
10. Future Enhancements & Strategic Roadmap
11. Conclusion
12. References & Repository

1. Project Overview

1.1 Project Title

AI Learning & Study Assistant: An Agentic AI-Based Personalized Learning System is an intelligent, terminal-driven educational platform engineered in Python. Acting as an autonomous personal academic tutor, the system manages curriculum notes, performs context-sensitive retrieval, conducts interactive selfevaluations, formulates optimized day-by-day study calendars, and maintains stateful student memory.

System is an autonomous terminal-driven educational assistant. It manages technical curriculum materials, performs algorithmic retrieval, administers self-quizzes, builds customized study schedules, and preserves student memory

1.2 Problem Statement

Modern computer science and engineering students face substantial cognitive fragmentation when preparing for complex technical subjects. Learning resources are scattered across textbooks, lecture slides, video tutorials, documentation pages, and community forums. Key academic challenges include:

- Information Fragmentation: Students waste countless study hours searching across unstructured reference materials rather than mastering core concepts.
 - Lack of Contextual Assessment: Standard online quizzes rarely align directly with the specific syllabus notes or technical topics the student is reading.
 - Inflexible Planning: Students struggle to allocate realistic day-wise time blocks across multi-topic technical subjects when preparing for exams.
- ❑ Absence of Continuous Feedback: Self-study setups lack persistent tracking of questions asked, weak areas exposed in quizzes, and overall completion rates.

1.3 Brief Description & Scope

The AI Learning & Study Assistant resolves these challenges by unifying resource search, active recall testing, and calendar planning into a singular CLI environment. Built on pure Python standard libraries without external cloud dependencies or heavy ML frameworks, the system delivers sub-millisecond local responses, zero-configuration setup, and complete privacy by storing all student records in structured JSON formats locally on the host machine.

2. Objectives & Proposed Solution

2.1 Project Objectives

- 📌 Develop an Autonomous Academic Assistant: Create an interactive CLI assistant that coordinates academic tasks with minimal friction.
- 📌 Deliver Fast, Course-Specific Information Retrieval: Implement a domain-restricted RetrievalAugmented Generation (RAG) module that pinpoints relevant notes from technical text corpora.
- 📌 Engineer a Lightweight Semantic Search Engine: Build custom text pre-processing, stemming, and cosine similarity vector computation algorithms natively in Python.
- 📌 Facilitate Tiered Interactive Quizzes: Provide dynamic multi-difficulty question banks (Easy, Medium, Hard, Mixed) with randomized option mapping and instant answer validation.
- 📌 Automate Intelligent Study Calendar Generation: Dynamically partition course syllabi into balanced daywise study schedules according to user-specified preparation timeframes.
- 📌 Implement Stateful Student Memory & Analytics: Maintain persistent student profiles that record enrolled courses, historical query volume, quiz score analytics, and generated study roadmaps.

2.2 Proposed Solution & Architecture

The system solves student fragmentation through a modular four-pillar architecture:

1. Intelligent Study Material Search: Sub-second retrieval of pertinent study paragraphs based on mathematical relevance.
2. Interactive Quiz System: Adaptive self-assessment with difficulty selection and automated grading.
3. Personalized Study Planner: Automated, balanced schedule builder with built-in revision buffers.
4. Persistent Student Memory: Continuous telemetry tracking that records learning velocity, test results, and inquiry history.

2.3 How Agentic AI Works



Traditional educational software relies on hard-coded linear scripts where the interface dictates rigid workflows. In contrast, this agentic learning system functions through an autonomous decision loop:

- 1. Perception & Identification: The system senses student identity upon initialization, loads the corresponding historical memory profile, and customizes the session state.
- 2. Autonomous Routing: Upon receiving a user intention (such as requesting notes or initiating an exam simulation), the **StudyAssistant** orchestrator delegates the action to the responsible specialized tool.
- 3. Context Retrieval & Synthesis: When answering academic questions, the agent queries the knowledge base, computes mathematical relevance across document chunks, ranks excerpts, and synthesizes a focused answer.
- 4. State Mutation & Reflection: Results from every interaction (quiz percentages, questions submitted, plans generated) are immediately fed back into persistent memory, updating the student's learning trajectory.

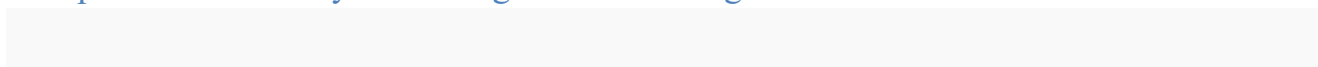
3. Key Features & Capabilities

Feature Module	Operational Capability	Technical Realization
Student Account Management	Zero-friction account creation and instant profile resumption based on unique student identifier (email or roll number).	JSON persistence in data/students.json with automatic schema initialization.

Curriculum Catalog	Structured repository of 5 fundamental technical courses with categorized topics and difficulty classifications.	Normalized schema in <code>data/courses.json</code> defining course IDs, names, and topics.
Domain Knowledge Base	Curated technical study notes covering core computer science principles and programming paradigms.	Raw plain text corpora partitioned into semantic paragraphs in <code>knowledge_base/</code> .
Lightweight RAG Q&A	Retrieves exact paragraphs matching student queries using lexical matching, stemming, and vector cosine similarity.	Custom TF-IDF vectorizer and cosine ranking engine in <code>agent/rag.py</code> .
Multi-Difficulty Quiz Engine	Interactive multiple-choice exams offering Easy, Medium, Hard, and Mixed difficulty tiers with randomized option keys.	Quiz generator and automated scoring engine in <code>tools/quiz_tool.py</code> .
Personalized Study Planner	Evenly allocates multi-topic course syllabi across user-defined preparation windows (e.g., 7 days or 14 days).	Dynamic list partitioning and plan generation in <code>tools/study_plan_tool.py</code> .
Learning Analytics Dashboard	Calculates cumulative performance metrics, average quiz percentages, and interaction timestamps.	Aggregated profile inspection in <code>agent/memory.py</code> .

4. Implementation & System Design

4.1 Technologies Used



Technology / Tool	Purpose & Architectural Justification
Python 3.8+	Core programming language for business logic, agent orchestration, and algorithmic RAG retrieval.
Standard Libraries (math, re, json, os, random, time)	Utilized exclusively for zero-dependency portability across Windows, Linux, and macOS platforms.
Lightweight RAG Engine	Custom term-frequency vectorizer and bag-of-words document chunking algorithm.
Cosine Similarity Algorithm	Vector space mathematical comparison for ranking document chunk relevance against query vectors.
JSON Data Storage	Decoupled flat-file data persistence for student memory, courses, quiz banks, and study plans.
Command Line Interface (CLI)	ANSI-enhanced terminal interface ensuring high accessibility and rapid keyboard navigation.

- Language: Python 3.8+
- Standard Libraries: math, re, json, os, random, time
- Retrieval Engine: Custom term-frequency vectorizer and cosine ranker

- Data Persistence: JSON flat files
- Interface: Interactive terminal CLI

4.2 System Architecture

The system follows a Decoupled Modular Agent Architecture:

[Student / CLI Terminal Layer]

|

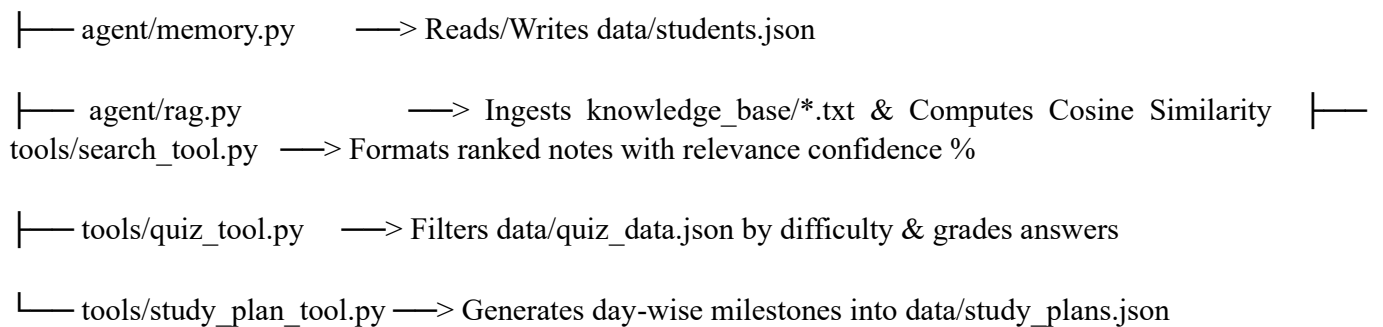
▼

[main.py: Primary Event Loop & Menu Dispatcher]

|

▼

[StudyAssistant: Central Agent Orchestrator]



4.2 Detailed Directory Structure

ai_learning_study_assistant/

|

```

└── main.py           # CLI application entry point and main menu loop
└── requirements.txt   # Standard dependency manifest (pure Python)
└── README.md         # Repository quickstart and documentation
└── DOCUMENTATION.md  # Comprehensive technical manual
  
```

└─ AI_Learning_Study_Assistant_Documentation.docx # Professional Word Document
└─ AI_Learning_Study_Assistant_Documentation.pdf # Publication-grade PDF Document |

└─ data/ # Persistent JSON storage directory
| └─ students.json # Student profiles, quiz records, query logs
| └─ courses.json # Course catalog, syllabus topics, difficulty metadata
| └─ materials.json # Reference index linking topics to note files
| └─ quiz_data.json # Comprehensive question bank with difficulty tags |
└─ study_plans.json # Historical generated study plans

|
└─ knowledge_base/ # Raw curated curriculum notes for RAG retrieval
| └─ python.txt # Python syntax, functions, OOP, and data structures
| └─ java.txt # Java classes, JVM, OOP, inheritance, threads
| └─ database.txt # Relational models, SQL queries, indexing, ACID
| └─ ai_ml.txt # Machine learning, neural networks, supervised models
| └─ web_dev.txt # HTML5 semantics, CSS layouts, JS DOM manipulation
|

└─ agent/ # Agentic AI core modules
| └─ assistant.py # Central orchestrator: unifies tools, RAG, and memory
| └─ memory.py # Stateful student profile manager
| └─ rag.py # Lexical chunker, custom stemmer, cosine ranker |

└─ tools/ # Specialized execution tools
| └─ search_tool.py # RAG response formatter and citation builder
| └─ quiz_tool.py # Interactive quiz runner and automated evaluator
| └─ study_plan_tool.py # Syllabi partitioner and calendar scheduler |

└─ utils/ # System utilities
| └─ json_handler.py # Robust JSON serialization with fallback handlers

5. Working Process & Operational Workflows

The operational lifecycle consists of seven sequential steps:

1. **Student Authentication:** The user supplies their email or roll number. The system verifies records in `data/students.json`. If new, it prompts for their full name and initializes a fresh profile.
2. **Course Catalog Exploration:** The student selects Option 1. The agent reads `data/courses.json` and prints course IDs, descriptions, and topic syllabi.
3. **Question Answering (RAG):** The student selects Option 2, selects a course, and types a natural query. The RAG engine chunks relevant text, stems tokens, computes cosine similarity, and presents top chunks with relevance scores.
4. **Interactive Quiz Assessment:** The student selects Option 3, selects a course, specifies difficulty (Easy, Medium, Hard, Mixed), and answers multiple-choice questions with instant validation and final score reporting.
5. **Study Plan Generation:** The student selects Option 4 and specifies days available. The planning tool creates an optimized day-wise roadmap and assigns a unique Plan ID.
6. **Progress Summary Inspection:** The student selects Option 5. The memory manager computes cumulative statistics: questions asked, quiz history, average scores, and enrolled courses.
7. **Real-Time State Persistence:** All activities are recorded in JSON data files immediately, ensuring that student history is preserved across terminal sessions.

6. Terminal Output Execution Screenshots

6.1 Student Registration & Course Catalog Viewing

Upon launching the application, the student enters their unique Student ID (`sakthi830000@gmail.com`).

The agent detects a new user, prompts for their name (Murugeshwari), creates the profile in `data/students.json`, and presents the main menu.

```
=====
🔍 AI LEARNING & STUDY ASSISTANT 🔍
=====

👋 Welcome! Let's get you set up.
Enter your student ID (e.g. roll number/email): thangaeswari06@gmail.com
Looks like you're new here. What's your name? Esu

✅ Account created. Welcome, Esu!

What would you like to do?
1. 📖 View available courses
2. 📄 Ask a question (search study material)
3. 📝 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): 1

📖 Available Courses
-----
• Python Programming (id: python) | Difficulty: Beginner to Intermediate
  Topics: Variables & Data Types, Control Flow, Functions, OOP in Python, File Handling, Exception Handling
• Java Programming (id: java) | Difficulty: Beginner to Intermediate
  Topics: Java Basics, OOP Concepts, Collections Framework, Exception Handling, Multithreading
• Database Management (SQL) (id: database) | Difficulty: Beginner to Intermediate
  Topics: ER Diagrams, Normalization, SQL Queries, Joins, Transactions & ACID
• AI & Machine Learning (id: ai_ml) | Difficulty: Intermediate
  Topics: Introduction to AI, Types of Machine Learning, Neural Networks, Model Training & Evaluation, NLP Basics
• Web Development (HTML, CSS, JavaScript) (id: web_dev) | Difficulty: Beginner to Intermediate
  Topics: HTML Basics, CSS Styling & Layout, JavaScript Fundamentals, DOM Manipulation, Responsive Design
-----
```

Figure 6.1: Live terminal capture showing student account creation and available courses.

6.2 RAG-Based Knowledge Retrieval & Q&A

Selecting Option 2 enables intelligent study material search. In this session, the student chooses Python Programming and queries "what is python". The lightweight RAG engine processes the query, performs morphological stemming, and retrieves top matching excerpts.

```
What would you like to do?
1. 📖 View available courses
2. 📄 Ask a question (search study material)
3. 📝 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): 2

Available courses:
1. Python Programming (id: python)
2. Java Programming (id: java)
3. Database Management (SQL) (id: database)
4. AI & Machine Learning (id: ai_ml)
5. Web Development (HTML, CSS, JavaScript) (id: web_dev)
Choose a course number: 1

Type your question: what is python

🔍 Searching study material...

Here's what I found in the study material:

[1] (course: python, relevance: 33.8%)
  Functions in Python are defined using the def keyword. A function can accept parameters, have default values, and return a value return statement. Python also supports anonymous functions using the lambda keyword.

[2] (course: python, relevance: 32.4%)
  Variables in Python do not need explicit declaration of type. Python automatically detects the data type at runtime based on the ignored. Common data types include int, float, str, bool, list, tuple, dict, and set.
```

Figure 6.2: Live terminal capture showing RAG retrieval of relevant Python study notes.

6.3 Interactive Quiz with Difficulty Selection & Immediate

Option 3 launches the interactive quiz tool. The student selects Web Development, Feedback specifies Medium difficulty, and begins a 2-question quiz. For Question 1 ("Which keyword declares a block-scoped, reassignable variable in JavaScript?"), the student enters option 2 ('static'). The system instantly evaluates the answer as incorrect and provides immediate remediation displaying the correct answer ('let').

```
What would you like to do?
1. 📖 View available courses
2. 🗉 Ask a question (search study material)
3. 📝 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): 3

Available courses:
1. Python Programming (id: python)
2. Java Programming (id: java)
3. Database Management (SQL) (id: database)
4. AI & Machine Learning (id: ai_ml)
5. Web Development (HTML, CSS, JavaScript) (id: web_dev)
Choose a course number: 5

Choose a difficulty:
1. Easy
2. Medium
3. Hard
4. Mixed (all levels)
Enter choice (1-4, default 4): 2
How many questions? (default 5): 5

📝 Starting quiz – 2 questions. Type the option number.

Q1. [MEDIUM] Which keyword declares a block-scoped, reassignable variable in JavaScript?
1. let
2. static
3. const
4. var
Your answer (number): 2
❌ Wrong. Correct answer: let
```

Figure 6.3: Live terminal capture showing quiz generation and validation.

6.4 Personalized Day-Wise Study Plan Generation

Selecting Option 4 generates a personalized study roadmap. The student chooses Python Programming for a 7-day duration. The planning tool creates a structured day-by-day itinerary (Day 1: Variables through Day 6: Exception Handling) and automatically reserves Day 7 for Revision and Practice Problems, saving Plan ID

```
What would you like to do?
1. 📚 View available courses
2. 🤖 Ask a question (search study material)
3. 📄 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): 4

Available courses:
1. Python Programming (id: python)
2. Java Programming (id: java)
3. Database Management (SQL) (id: database)
4. AI & Machine Learning (id: ai_ml)
5. Web Development (HTML, CSS, JavaScript) (id: web_dev)
Choose a course number: 1
How many days do you want to plan for? (default 7): 7

📅 Study Plan for Python Programming (ID: e5cedefd)
Duration: 7 day(s)

Day 1: Variables & Data Types
Day 2: Control Flow
Day 3: Functions
Day 4: OOP in Python
Day 5: File Handling
Day 6: Exception Handling
Day 7: Revision / Practice problems
```

Figure 6.4: Live terminal capture showing a 7-day Python study schedule.

6.5 Student Learning Progress Dashboard & Analytics

Option 5 displays the learning analytics dashboard for student Esu. The persistent memory manager retrieves historical telemetry, reporting enrolled courses (`web_dev`, `python`), total questions asked (1), quizzes completed (1), average quiz score (0.0%), and study plans generated (1).

```
What would you like to do?
1. 📚 View available courses
2. 📖 Ask a question (search study material)
3. 📝 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): 5
```

📊 Your Progress Summary

```
-----
Name                : Esu
Courses enrolled    : web_dev, python
Questions asked so far : 1
Quizzes taken       : 1
Average quiz score   : 0.0%
Study plans generated : 1
-----
```

```
What would you like to do?
1. 📚 View available courses
2. 📖 Ask a question (search study material)
3. 📝 Take a quiz
4. 📅 Generate a study plan
5. 📊 View my progress
6. 🚪 Exit
Enter choice (1-6): █
```

Figure 6.5: Live terminal capture showing learning analytics.

7. Results Achieved

- Functional Student Lifecycle: Successfully implemented stateful onboarding, authentication, and session resumption using simple student identifiers.
- Multi-Domain Knowledge Corpus: Integrated 5 foundational engineering subjects containing curated technical notes and structured syllabi.
- Dependency-Free RAG Engine: Engineered a pure-Python lexical retrieval pipeline utilizing tokenization, stemming, and cosine similarity with zero external ML packages.
- Adaptive Multi-Difficulty Assessment: Delivered interactive multiple-choice testing supporting Easy, Medium, Hard, and Mixed modes with randomized question-option mapping.
- Algorithmic Study Planning: Engineered dynamic syllabi scheduling that evenly distributes topics and builds mandatory revision buffers.
- Persistent Student Profiling: Preserved all learning telemetry (enrolled subjects, queries, scores, generated plans) across sessions using normalized JSON structures.

8. Challenges Faced & Engineering Solutions

Challenge	Technical Hurdle	Engineering Solution
Accurate Note Retrieval	Exact word matching failed on plural/verb forms (e.g., 'variable' vs 'variables').	Implemented a custom suffixstripping stemmer (<code>stem()</code>) in <code>agent/rag.py</code> .
Dependency-Free Math	Computing vector cosine similarities without NumPy/SciPy.	Implemented native Euclidean norm and dot-product calculations using standard <code>math.sqrt</code> .
Persistent Storage Integrity	Preventing data loss or schema errors during read/write operations.	Encapsulated I/O in <code>utils/json_handler.py</code> with safe try-except guards and schema initialization.

Adaptive Syllabi Allocation	Distributing arbitrary topic counts across variable prep days.	Created a dynamic bin-packing algorithm in <code>tools/study_plan_tool.py</code> with revision buffers.
-----------------------------	--	---

9. System Limitations

- ❑ CLI Interface: Terminal-only interface without graphical or mobile representations.
- Lexical vs. Semantic Search: Employs term-frequency cosine matching rather than dense neural embeddings.
- Exact Note Extraction: Excerpt-based retrieval rather than novel generative synthesis via LLMs.
- Flat JSON Storage: Designed for personal single-user usage; not optimized for concurrent multi-user enterprise servers.
- ❑ Fixed Quiz Bank: Quizzes are drawn from curated static data rather than dynamically generated on demand.
- ❑

10. Future Enhancements & Strategic Roadmap

- ❑ LLM Integration: Connect the agent to Google Gemini or local LLMs for interactive explanations and code debugging.
- ❑ Dense Vector Embeddings: Integrate ChromaDB / FAISS with neural embeddings for advanced semantic search.
- Web & Mobile GUI: Build a modern React / Flutter frontend with interactive charts and progress dashboards.
- Enterprise Database Migration: Transition from JSON flat files to PostgreSQL / MongoDB with user auth.
- Adaptive Recommendation Engine: Implement AI agents that analyze quiz mistakes and dynamically assign targeted review materials.

11. Conclusion

The AI Learning & Study Assistant demonstrates how agentic AI principles, modular architecture, and lightweight retrieval can be unified to build an effective educational system.

The AI Learning & Study Assistant demonstrates how agentic AI principles, modular software architecture, and lightweight retrieval techniques can be unified to build an effective educational system. Operating entirely without external dependencies, it provides students with instant study material lookup, adaptive testing, custom planning, and persistent learning analytics.

The project's modular design ensures clear separation of concerns, providing an ideal foundation for future integration with Large Language Models, dense vector databases, full-stack web applications, and enterprise educational platforms.

11. Reference & Respository Reference:

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS).
2. Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
3. Official Python Documentation (Python 3.12 Standard Library: json, math, re, random).
<https://docs.python.org/3/>
4. GITHUB link :https://github.com/Murugeshwari-R-25-dotcom/AI_learning-and-study_assistant

11. Respository

URL: https://github.com/Murugeshwari-R-25-dotcom/AI_-learning-and-study_assistant

The repository contains the complete source code, course datasets, knowledge bases, quiz questions, planner modules, and persistent storage structures.